SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0309	**Course Title:**	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	**Assessment:**	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	**Total Marks:**	100
CO2 Statement:	*Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.*		
Student Name:	A. Sathwika Reddy	**Reg. No.:**	192525274
Section / Batch:	AIML	**Date:**	17/07/2026

Code of Conduct

I A. Sathwika Reddy - 192525274 *(Name / Reg No)*
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Sathwika Reddy

Instructions

- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
	GRAND TOTAL:	**100 Marks**	

Annexure A – Analytical Problem Solving

S.No	REG NO	NAME	ANALYTICAL PROBLEM SOLVING
1	192525274	ARATIPALLI SATHWIKA REDDY	1. Perform the following operations in LIFO data structure having an array size of 5. Push(45), Push(22), Push(11), Push(77), Pop(), Push(44), Push(33), Pop(), Pop(),Pop(). Print the peak element of the data structure and its index position. With a neat sketch explained in detail. 2. How do you implement Queue data structure using Single Linked List? With a neat diagram discussing Enqueue and Dequeue operations using SLL.
2	192511003	ARREDDULA LIKITHASRI	1. Perform the following operations in Circular Queue having a size of 5. Enqueue(20), Enqueue(10), Enqueue(30), Dequeue(), Enqueue(80), Dequeue(), Enqueue(90), Enqueue(100), Dequeue(), Dequeue(), Enqueue(60), Enqueue(90). Finally display the front and rear elements with its positions. Discuss in detail. 2. Implement Binary searching Algorithm for searching an element and its position having the following 10 elements. Elements are 66, 111, 212, 323, 432, 543, 643, 649, 777, 888. Case 1: Key element = 111, Case 2: Key element=888, Case 3: Key element = 999. Analyse the concept in detail.
3	192525224	BHAVANASI SATHVIKA	1. Pictorially insert an element at 3rd position in a doubly linked list, having 6 elements in the list and delete the element at 2nd position from the list. Implement and analyse it. 2. Perform the following operations using stack. Assume the size of the stack is 5 and have a value of 22, 55, 33, 66, 88 in the stack from 0 position to size-1. Now perform the following operations: 1) Invert the elements in the stack, 2) Pop(), 3) Pop(), 3) Pop(), 4) Push(90), 5) Push(36), 6) Push(11), 7) Push(88), 8) Pop(), 9) Pop(). Draw the diagram of the stack and illustrate the above operations and identify where the top is?
4	192511402	BHUVANESHWARA N J	1. Convert the following infix expression into a post fix expression. B - (C/D + (E % F* G) / H) * J. Also, evaluate the given postfix expressions. a) 9 3 8 4 /*+ b) 6 2 + 6 3 2. Illustrate the queue operation using following function calls of size = 5. Enqueue(21), Enqueue(67), Enqueue(89), Dequeue(), Enqueue(15), Enqueue(30), Enqueue(52), Dequeue(), Dequeue(), Dequeue(), Dequeue().

38	192511310	VAIBHAVA LAKSHMI K	1. Two algorithms solve the same problem. Algorithm A uses recursion, while Algorithm B uses an explicit stack. Compare both approaches in terms of memory usage, execution flow, and risk of stack overflow. 2. A stack has a capacity of 5 elements. Explain what happens if the following operations are executed: Push(A), Push(B), Push(C), Push(D), Push(E), Push(F). Identify the error and suggest two possible solutions.
39	192524458	VASANTHAN S	1. Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately. 2. A queue implemented using an array report "Queue Full" even though some positions are empty. ➢ Analyze why this occurs. ➢ Explain how a circular queue solves this problem.
40	192524083	YARRAPPAREDDYG ARI SAI GOPAL REDDY	1. Consider the singly linked list: 10 → 20 → 30 → 40 → 50 Delete the node containing 30. Explain how the links change after deletion. 2. Given two sorted linked lists: L1: 1 → 3 → 5 → 7 L2: 2 → 4 → 6 → 8

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1) Given

* Array size = 5
* operations :

push(45), push(22), push(11), push(77), pop(),
push(44), push(33), pop(), pop(), pop

* A stack follows the LIFO principle.
The element inserted last is removed first.

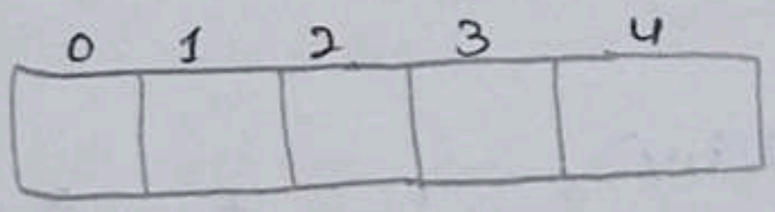
We use

* Top = -1 initially
* Array size = 5
* Index positions: 0, 1, 2, 3, 4

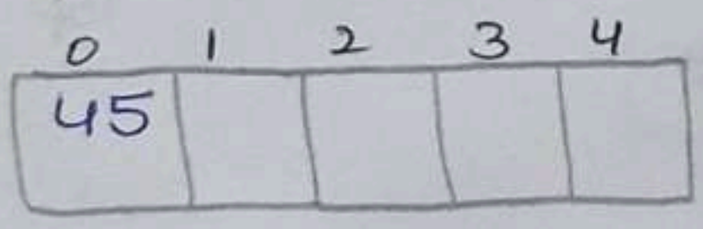
Indial condition

Stack: Empty

TOP = -1

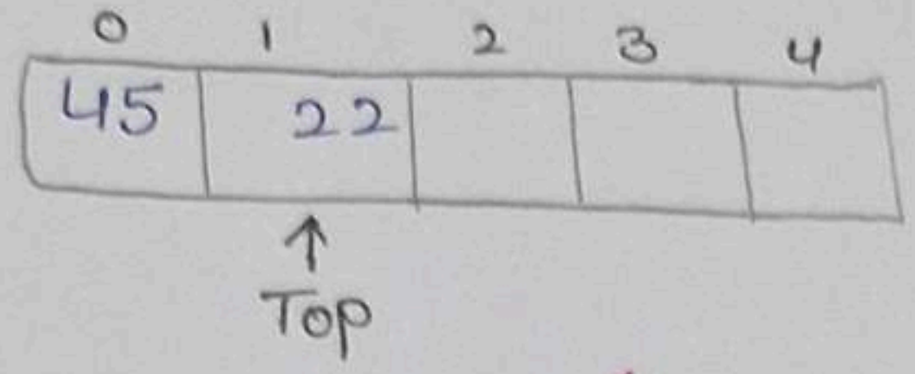


push(45)

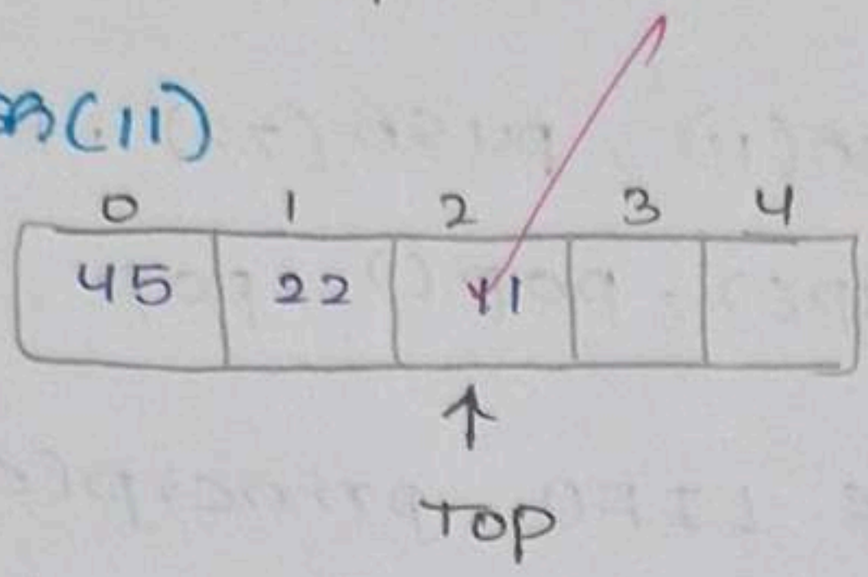


45 is inserted and TOP becomes 0

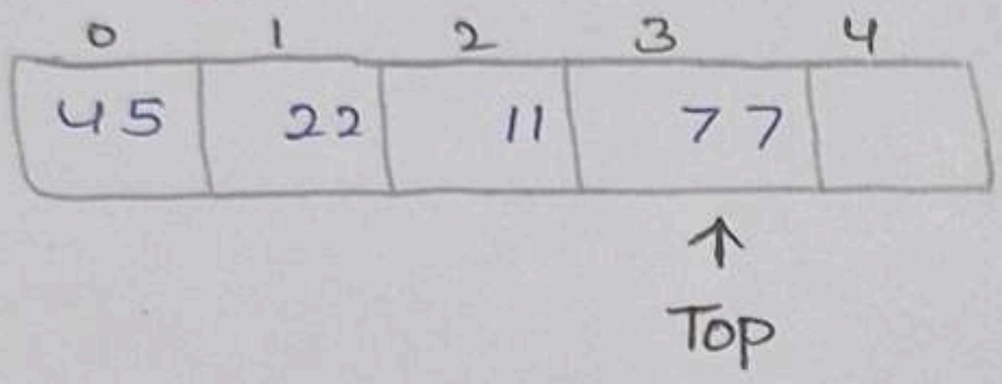
push(22)



push(11)

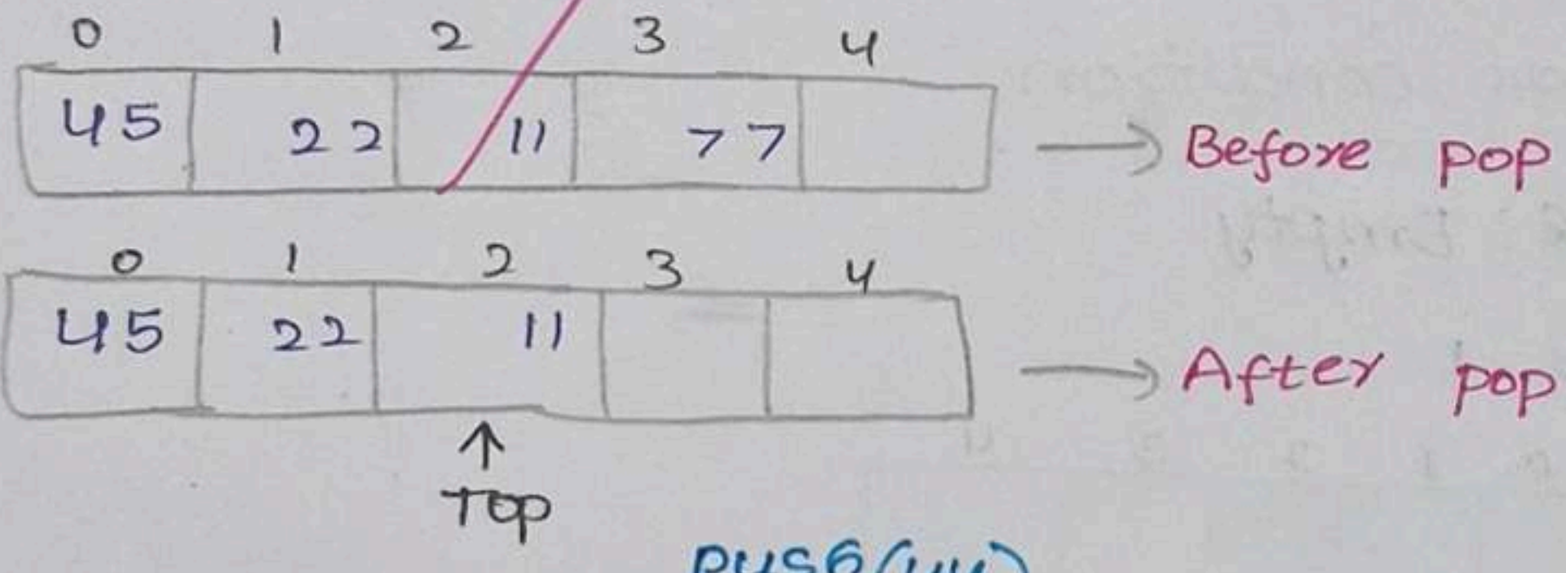


push(77)

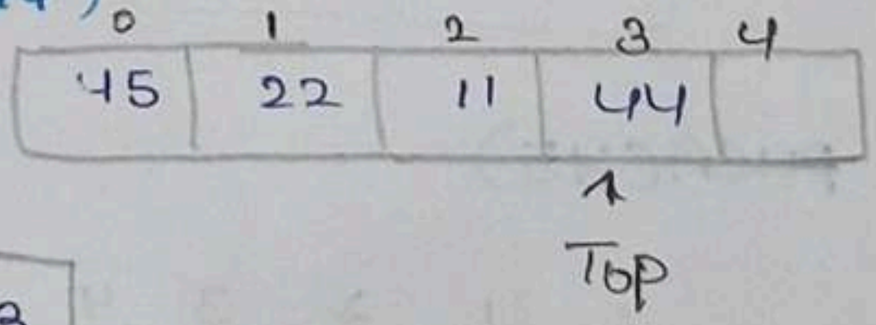


pop()

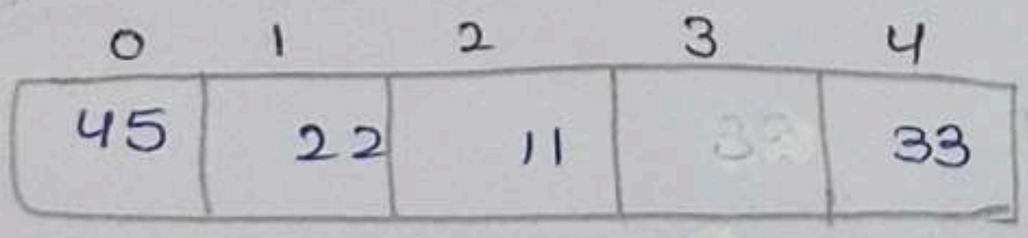
The top element 77 is removed



push(44)

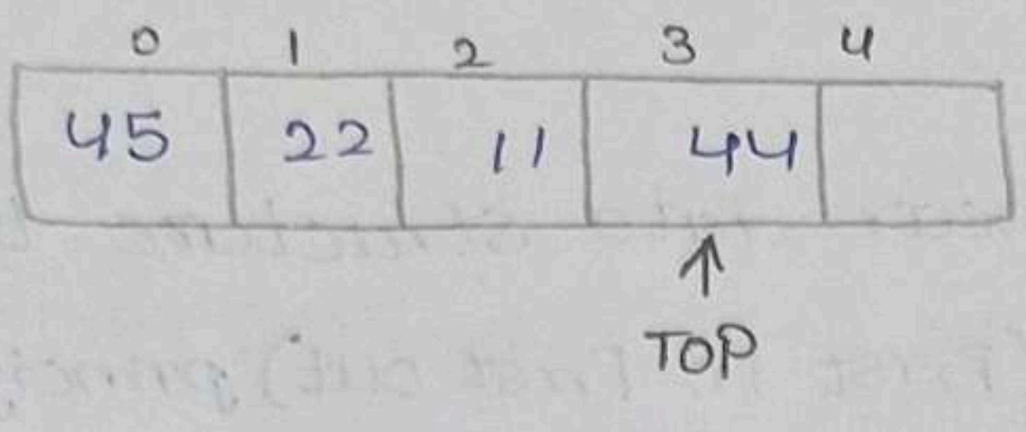


push(33)



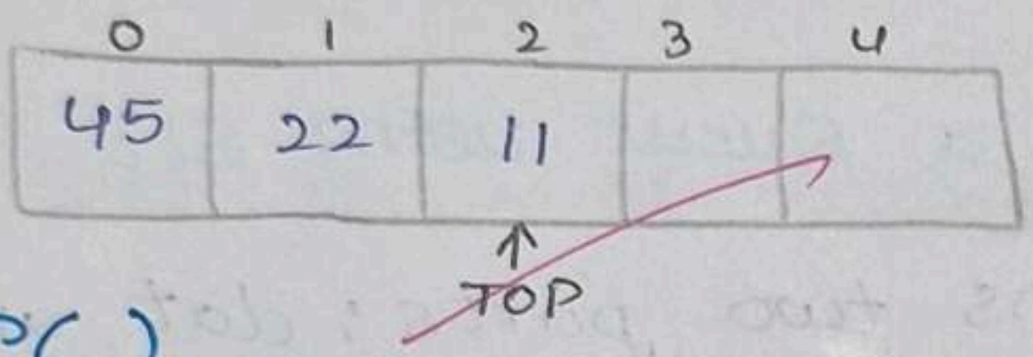
pop()

The element 33 is removed



pop()

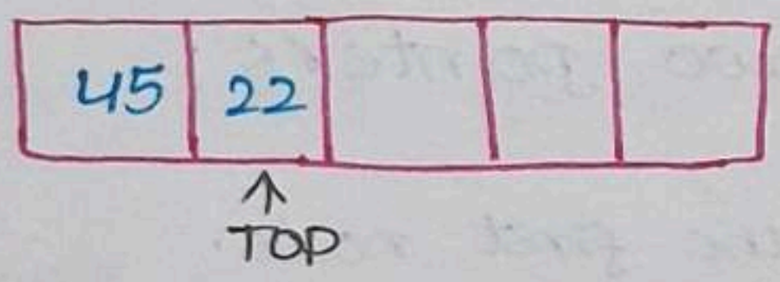
The element 44 is removed



pop()

The element 11 is removed

Final stack



∴ peak/Top element = 22

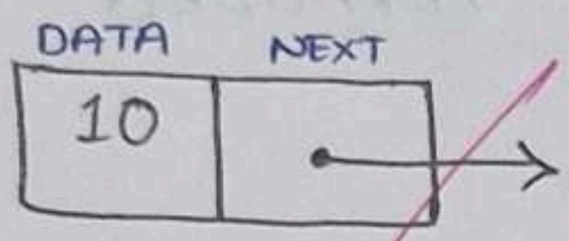
Index/position = 1 (0-based indexing)

2) Queue Data Structure usingly Singly Linked List

* A Queue is a linear data structure that follows the FIFO (First in First out) principle in queue, insertion is performed at rear and deletion is performed at front.

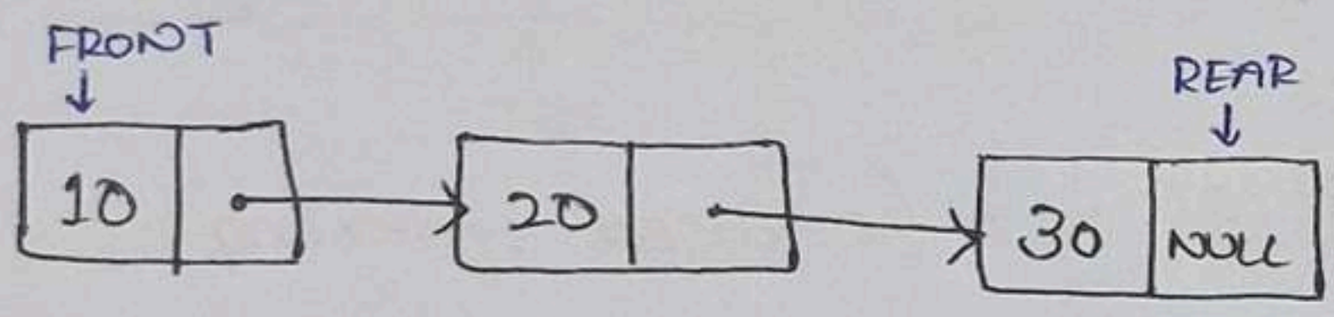
Representation of a Queue using SLL

* Each node contains two parts: dat and next pointer



A queue maintains two pointers:...

* FRONT - points to the first node.
* REAR - points to the last node.



1) Enqueue Operation

Enqueue operation means inserting an element into the open, queue. The new node is always inserted at the REAR

Steps :-

1) Create a new node.

2) Store the data in the new node.

3) Set Newnode → next = NULL

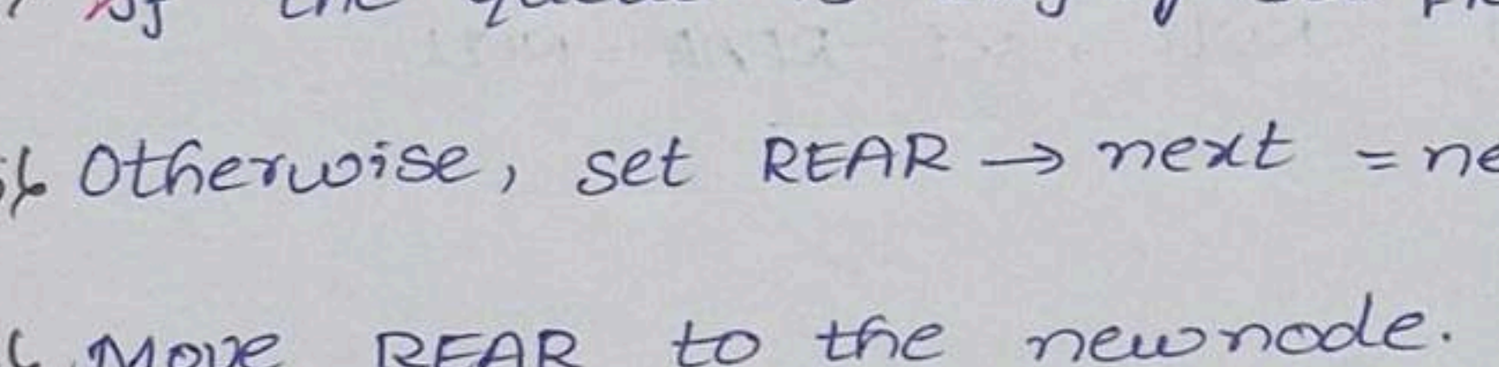
4) If the queue is empty, set FRONT = REAR = newnode.

5) Otherwise, set REAR → next = newNode

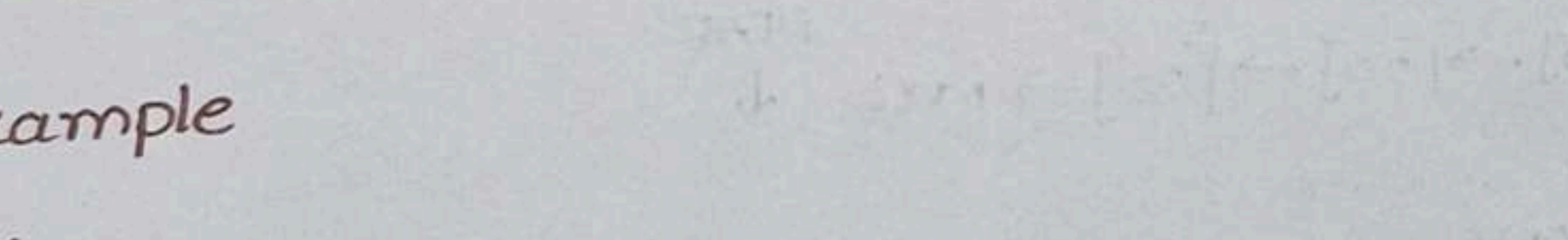
6) Move REAR to the newnode.

Example

Before



After



Dequeue Operation

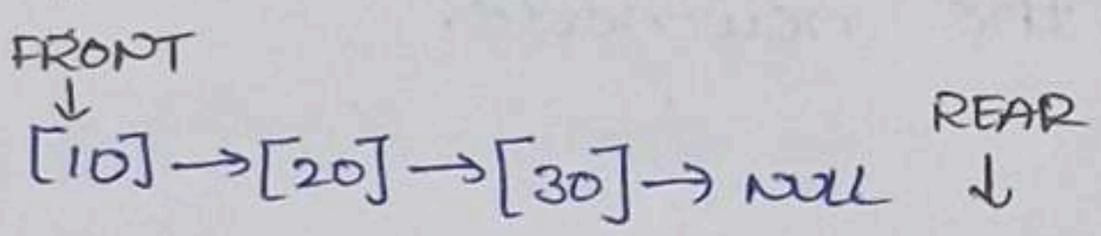
Dequeue means deleting an element from the queue. The element is alway deleted from FRONT

Steps :-

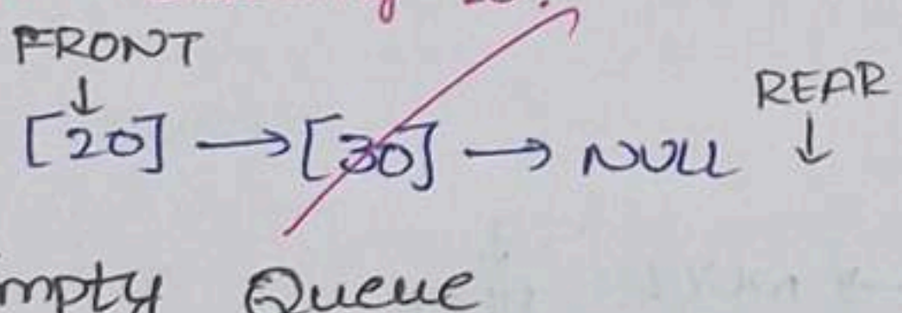
1) Check whether FRONT == NULL
2) If true, the queue is empty
3) otherwise, store the front node in temporary pointer.
4) move FRONT to FRONT → next.
5) Delete the previous front node.
6) If FRONT == NULL, set REAR = NULL

Example :-

Before:



After deleting 10:



Empty Queue

FRONT = NULL , REAR = NULL

Conclusion :- Queue can be implemented using a singly Linked list by maintaining two pointers, FRONT and REAR.